Because the replicas aren't yet instantiated, there is a high chance that we will lose data if anything happens to Node A. Due to this risk, the cluster's health status is set to YELLOW.

We know that all the shards are on a single node, and if this node crashes for any reason, we will lose everything. To avoid data loss, we decide to start a second node to join the existing cluster. Once the new node (Node B) is created and added to the cluster, Elasticsearch distributes the original three shards as follows:

1. Shard 2 and Shard 3 are removed from Node A.
2. Shard 2 and Shard 3 are added to Node B.

This move distributes our data across the cluster by adjusting the shards, as figure 3.11 shows.

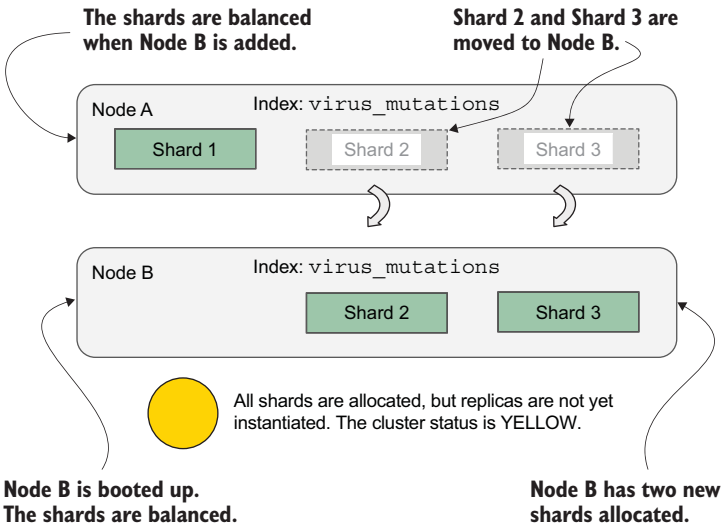


Figure 3.11 Shards are balanced onto the new node, but replicas aren't assigned yet (YELLOW status).

As soon as the shards are distributed after we add Node B, the cluster status turns YELLOW.

> **NOTE** Elasticsearch exposes the cluster's health via an endpoint: `GET _cluster/health`. This endpoint fetches the cluster's details, including the cluster name, status of the cluster, number of shards and replicas, and so on.

When the new node springs into life, in addition to distributing shards, replicas are instantiated. A clone of each shard is created, and data is copied to these clones from the respective shards. As mentioned earlier, the replicas do not sit on the same node as the primary shard. Replica 1 is a copy of Shard 1, but it is created and available on Node B. Similarly, Replica 2 and Replica 3 are copies of Shard 2 and Shard 3, respectively, which are on Node B, but the replicas are available on Node A (figure 3.12).